Graphical User Interface for the Arduino and Raspberry Pi v1.5

© 2026 MicroPhonon LLC. Licensed under Creative Commons



Description: This software package provides a graphical user interface for two-way communication between the AquaPing acoustic leak sensor and a personal computer. The AquaPing sends and receives data bytes as a slave on the serial I2C protocol, so intermediary hardware is required when using a laptop or desktop computer. Widely available and inexpensive Arduino and Raspberry Pi microcontroller boards can perform the function of I2C master. The master also routes sensor data to and from a PC. Because of the emphasis on low-power IOT applications using a 3V3 power bus, the code has been developed for two specific hardware types:

- Arduino Nano 33 IOT that is loaded with the arduino_gui.ino firmware
- Raspberry Pi Pico that is loaded with the raspi_gui.ino firmware

Minor modifications are needed to implement wireless Bluetooth or WiFi as well as other Arduino and Raspberry Pi variants. Appropriate edits can be determined by a competent LLM.

Two customizable software files are provided: i) aquaping_gui.py is a python/tkinter application that runs on a PC and ii) one of the hardware-specific .ino files listed above that is coded in C. Sensor data is read and displayed on the GUI; the monitoring parameters can also be configured. The raspi_gui.ino firmware has been ported to micropython; see below.

The software is open-source and should work with any operating system. Refer to the user manual for detailed description and interpretation of the sensor data stream and configuration of the control parameters. A text-only CLI for serial terminal communication is also available.

Installation. The AquaPing firmware remains unchanged. The GUI requires python and the pyserial package installed on the PC. On Linux, the program must be given executable permission and can be run on the CLI as a batch file by entering ./aquaping_gui.py or python3 aquaping_gui.py. The Linux user must also be a member of the dialout group.

The free Arduino IDE is used to flash the .ino firmware onto the master device via USB serial. The Raspberry Pi Pico uses the Earle Philhower Raspberry Pi Pico/RP2040 board package. It can be installed by following these instructions:

https://learn.adafruit.com/using-raspberry-pi-pico-pico-2-with-arduino/rp2040-arduino-ide-setup

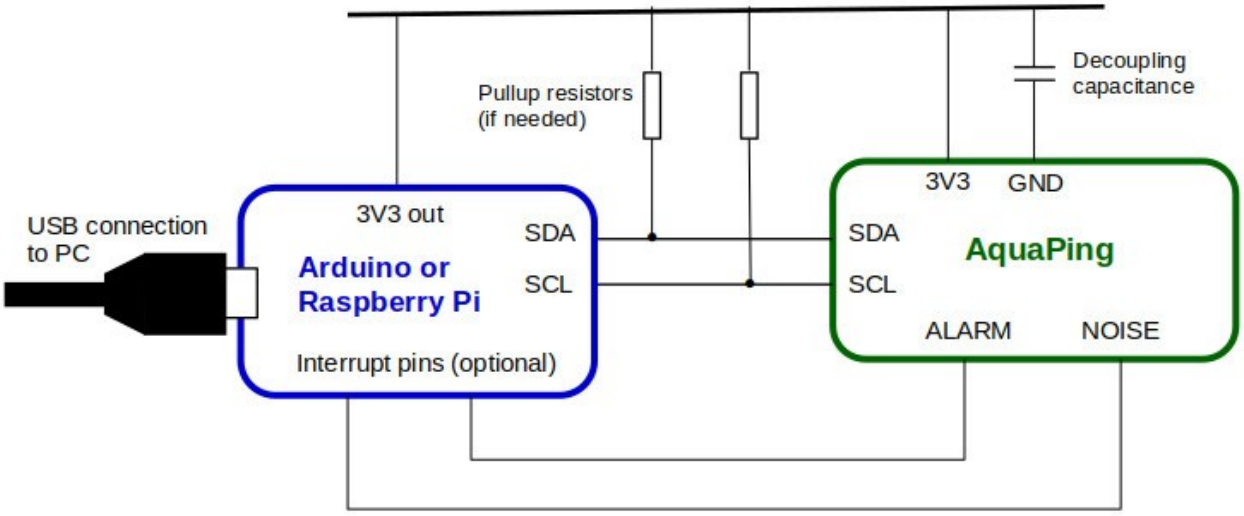
The two files arduino_gui.ino and raspi_gui.ino differ primarily by the pin assignments.

The file raspi_gui.py is also available for users who prefer micropython when working with the Raspberry Pi. Setup the RP Pico with the necessary .uf2 file from micropython.org. Then open raspi_gui.py with an IDE like Thonny and save it as main.py to the RP Pico to make it automatically load at boot. It should behave the same as the .ino setup and allow communication with the sensor using aquaping_gui.py.

Hardware. The AquaPing is designed to run from a *quiet* DC power source in the range 1.8–3.3V. Suitable supply voltage can be obtained directly from the Arduino Nano 33 IOT and Raspberry Pi Pico; consult the board schematic for the designated +3V3 and GND pins. The default power-save-mode of the RP Pico, however, produces excessive ripple on the 3V3 line that will send the front-end amplifier stages of the AquaPing into oscillation. The raspi_gui.ino firmware switches it into a quieter PWM mode. In both cases, placing a large electrolytic decoupling capacitor across the 3V3 power bus is mandatory. The AquaPing will be damaged if it is exposed to 5 Vdc and higher. If only 5V is available, an inexpensive voltage regulator or step-down converter must be used.

Caution: The Nano 33 IOT is known to generate noise transients on the power bus when configured for wireless operation, i.e. during the brief intervals when the radio is transmitting. Extra steps may be needed to prevent disruption of AquaPing operation.

A USB Micro-B cable connects the PC to the I2C master hardware at a baud rate of 9600. Jumper wires link the master to corresponding I2C ports on the AquaPing. On the Nano 33 IOT, the SDA and SDL pins are A4 and A5, respectively. The RP Pico uses GP4 and GP5, respectively. Pullup resistors are already present on the Nano 33 IOT, but the RP Pico does not have this feature. External pullup resistor values of 4.7k (not critical) are a good choice for a clock speed of 100 kHz; the pullup voltage is also 3V3. Use of a higher pullup voltage will likely damage the AquaPing. A conceptual wiring diagram is shown below.

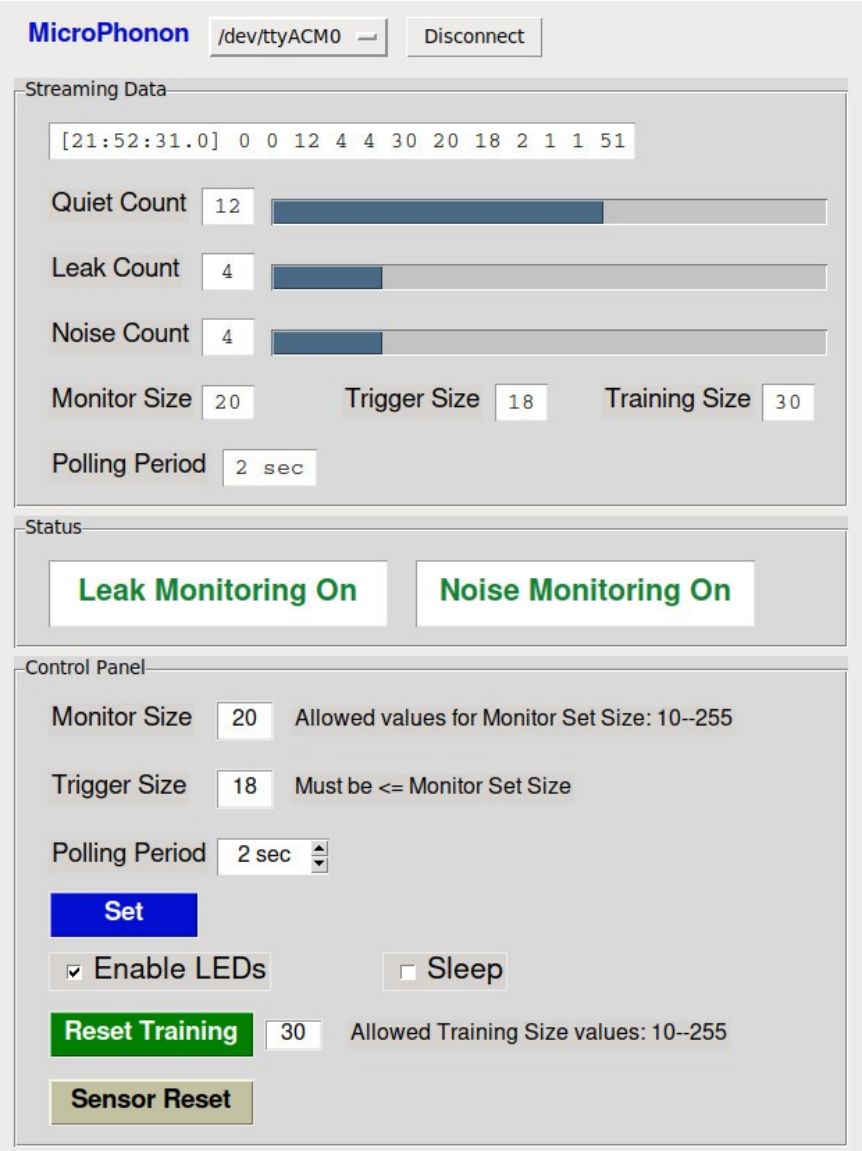


Immediate alerts (i.e. without waiting a full polling period to perform an I2C read) are possible by wiring the ALARM and/or NOISE pins on the AquaPing to interrupt ports as configured on the master. This can be useful when the master is set for very long delay between reads. The latency is then determined by the environmental polling period of the AquaPing. The interrupt lines also allow the sensor to be optionally deployed without wiring the I2C interface. The .ino file for the Nano 33 IOT is coded for interrupts on pins D3 and D2 for ALARM and NOISE, respectively. The RP Pico firmware sets the interrupts on GP3 and GP2, respectively. Modify as needed.

Operation: On initial power up or reset, the AquaPing will enter Training Mode as described in the user manual. The master will begin retrieving data from it via I2C at a 1 Hz rate. The surface-mount LED located at the corner of the master board blinks to signal each transfer. The master polling rate is adjusted by changing the polling_interval constant in the .ino firmware; default is 1000 (ms).

It is important to understand that the master and slave (AquaPing) are operating independently and asynchronously. Acoustic data transport from the ADC along with I2C read/write operations are handled without CPU intervention by Direct Memory Access (DMA) on its MSP430 microcontroller. The acoustic monitoring interval of the AquaPing and the polled I2C read events are entirely separate operations and user-configurable.

Upon launching aquaping_gui.py, the user is presented with a multi-panel GUI shown below. Available serial ports should be automatically identified. On Linux, this may look like /dev/ttyUSB0 or dev/ttyACM0; they will be shown as COM ports on Windows. Press the button to connect. If automatic serial port detection fails, modify the .py code to specify the port manually.



When the PC connects to the Arduino or Raspberry Pi, the Streaming Data panel is continuously updated. A time-stamped, 12-number data stream is read from the AquaPing and shown in decimal format. Seven of the 12 numbers are parsed and displayed individually. As a visual aid, three horizontal progress bars accompany the Quiet, Leak, and Noise counts. Full-scale of each bar is the Monitor Size.

The Status panel displays short messages to indicate if the sensor is training or in monitoring mode. Alerts appear when a potential leak or noisy environment is detected. The alert conditions are thoroughly described in the user manual.

The Control Panel allows the user to configure the AquaPing. When the GUI is launched, default values for the different parameters are shown. The size of the Monitor data set and number of counts to reach Threshold can be changed by entering new values. Nine different Polling Periods are available in the range 1–30 seconds[1]. Modify these values as desired and press the **Set** button. This resets all data counts. A checkbox toggles LED operation on the AquaPing PCB; an adjacent checkbox puts the sensor in a sleep state where all monitoring stops and power consumption is minimized. The **Reset Training** button forces the sensor to reacquire a background at a 1 Hz rate with the specified number of samples. Pressing **Sensor Reset** restores all parameters to their default values and initiates a new training session.

This is experimental software. Unexpected behavior can sometimes be fixed by momentarily unplugging the USB cable and re-connecting.

1 The Polling Period is *not* the period of the I2C data reads, which can only be changed by directly editing the .ino file.